

Inventory AI

Agents and Slash Commands Guide

This guide explains every AI agent in Inventory AI: why it exists, what it does, which tools it uses, and how it fits the owner approval workflow. It also documents every slash command available in Agent Inbox chat (/stock, /forecast, /scan, and so on), including examples, roles, and what appears in the UI.

Inventory AI is inventory management for a small retailer (roughly 50-500 SKUs). Phase 1 is the system of record: catalog, stock ledger, purchase orders, suppliers. Phase 2 adds demand forecasts from sale history. Phase 3 adds agentic AI: agents propose; people approve. Agents never post stock, never send email, and never create purchase orders without owner approval.

Generated from docs/ORCHESTRATOR.md, AGENTS.md, WORKFLOW.md, and PLAN.md.

1. How the Chat Orchestrator Works

All agent work starts in Agent Inbox chat. You type a slash command or plain English. The orchestrator is a LangGraph pipeline with seven nodes: understand, validate, plan, approve_plan, dispatch, aggregate, and reply. Run state is stored on agent_runs with plan_data, state_data, and events_data. The browser receives Server-Sent Events (SSE) so you see thinking, a plan checklist, step progress, typed result cards, and a summary.

Pipeline stages

- understand: Slash commands map directly to intents. Free text goes to the LLM, which returns intents with confidence. Names like supplier or SKU are resolved to tenant ids in code.
- validate: Staff cannot run owner-only intents. Low confidence or ambiguous text becomes a clarification card. Unknown requests are refused with the supported command list.
- plan: Single intents use a fixed routing table. Compound requests (e.g. /scan /reorder /email) let the LLM propose a DAG, which code validates for cycles, unknown agents, and write-before-read errors.
- approve_plan: Any plan with a write step (draft PO or draft email) pauses until the owner clicks Run, Edit, or Cancel. Read-only plans run immediately.
- dispatch: Steps run in dependency order; independent reads may run in parallel. Each step returns a TypedResult (structured data), not free-form prose.
- aggregate: The LLM writes a short summary. If it invents a number not in the typed results, code replaces the summary with a template.
- reply: Emits token, card, and done events to the chat UI.

Core safety rules (all agents)

- No LLM arithmetic: on-hand, forecasts, reorder quantities, and money come from service tools only.
- No direct writes: write tools insert agent_suggestions and supplier_messages drafts only.
- Tenant scope: business_id always comes from the JWT, never from the model.
- Untrusted text is DATA: product names and tool JSON cannot change instructions or tool allowlists.
- Grounding: summaries and explanations that contain numbers or dates not in evidence are replaced with templates.

2. Forecast Agent

Use case

A shop owner needs to see expected demand before buying stock. The forecast agent answers questions about on-hand levels and projected sales. It powers the `/stock` and `/forecast` commands in chat and complements the Insights page charts.

Why it was added

Phase 2 added demand forecasting from sale movements (read-only, no purchase orders). Phase 3 wrapped that service in an agent so chat can show forecasts and stock in one place. Later agents (replenishment, exception monitor) consume the same forecast numbers.

Tasks

forecast

Runs demand analysis for one SKU or all products. Calls `run_forecast` (backtests `daily_average` vs `seasonal_naive` over the last 7 days, picks lower WAPE), `get_forecast`, `get_history`, and `get_forecast_accuracy`. Returns a `forecast_chart` card with per-SKU: history units, forecast units, trend (up/down/flat), chosen model, backtest WAPE, naive WAPE, confidence, and caveats (`low_history`, `intermittent_demand`). The LLM may add one grounded interpretation sentence; invented numbers are replaced with a template.

get_stock

Returns on-hand by product and location via `get_stock`. Shows a `stock_table` card with SKU, name, on-hand, reorder point, and low/out status. Read-only.

Tools used

- `get_stock`, `get_forecast`, `get_history`, `run_forecast`, `get_forecast_accuracy`

What it does NOT do

- Does not create purchase orders or post stock movements.
- Does not invent quantities; all numbers come from the forecast and inventory services.

3. Exception Monitor Agent

Use case

Small shops miss stockouts, overdue POs, demand spikes, and data errors because nobody scans every SKU daily. The exception monitor runs deterministic detectors and surfaces findings in chat (/scan) and via a nightly job (owner-configurable in Settings).

Why it was added

Phase 4 (exception handling) turns silent drift into inbox work. Detectors run in pure Python; the LLM only ranks playbook actions from an allowlist. Findings persist to the exceptions table with structured evidence for the explainer agent.

Task: scan

- 1) scan_detectors runs all detectors for the tenant.
- 2) The LLM ranks candidate playbook actions per finding (reorder_now, expedite, alternate_supplier, transfer_stock, count_stock, ignore).
- 3) record_exception_actions upserts open exceptions and may create generic agent_suggestions when approval is needed.

Returns an exception_list card. Orchestrator treats /scan as read-only (no plan pause), even though persistence happens inside the agent tools.

Detectors (code, not LLM)

- stockout_risk: days of cover (on_hand / daily_demand) below lead time; critical if on-hand is 0 with demand.
- overstock: more than 90 days of cover (180 = high severity).
- demand_spike / demand_drop: last 7 vs prior 7 ratio or robust z-score.
- supplier_delay: overdue approved/sent POs; or supplier reliability score below 0.7 with 2+ overdue.
- data_anomaly: negative on-hand, duplicate sales in the same minute, large unexplained adjustments.
- chase_no_reply: sent supplier chase with no reply after chase_followup_days (default 3).

Evidence stored

Each exception stores detector evidence plus reason_codes, forecast method/model, lead time, and supplier reliability when known. Generic suggestions copy evidence into payload.extra.

4. Replenishment Agent

Use case

Turn forecast and on-hand into concrete buy quantities and optional draft purchase-order suggestions. Answers `/reorder` (read-only recommendations) and `/draft-po` (owner write after approval).

Why it was added

Phase 3 procurement closes the loop from forecast to action with a human gate. Quantities use the deterministic formula in `replenishment.py`: $\text{needed} = \max(\text{reorder_point}, \text{forecast_units}) + \text{safety_stock}$; $\text{recommended_quantity} = \text{needed} - \text{on_hand}$. The LLM never computes these numbers.

Task: recommend

Calls `reorder_recommendations`, `get_supplier_reliability`, and `get_open_pos`. The LLM may filter which SKUs to include, prefer an alternate supplier when the preferred one is late, and set confidence. Returns a text card with lines, reason codes, and evidence per SKU. Does not write.

Task: draft_po

Same reads as `recommend`, then `create_po_suggestion` per supplier after `validate_po_proposal` passes. Stores `draft_po` suggestions with reason codes (`LOW_COVER`, `FORECAST_UP`, `SUPPLIER_LATE`, `ALTERNATE_SUPPLIER`), evidence (`on_hand`, `forecast`, `lead time`, `reliability`, `unit cost`), and confidence. Skips SKUs already covered by open POs. Requires owner role and plan approval before dispatch.

Reason codes

- `LOW_COVER`: on-hand below needed level.
- `FORECAST_UP`: forecast exceeds reorder point.
- `SUPPLIER_LATE`: chosen or preferred supplier reliability below 0.7 or has overdue POs.
- `ALTERNATE_SUPPLIER`: chosen supplier is not the preferred link.

Approval outcome

Owner approves a `po_suggestion` card or Approvals tab item. That creates a draft purchase order via the same service a person uses. Reject stores a reason on the suggestion and `audit_log`.

5. Purchase Guardrail (guardrail agent)

Use case

Block unsafe or malformed purchase proposals before they become suggestions or orders. Every write path (draft_po) ends with guardrail.review.

Why it was added

Phase 6 guardrails: agents share the ledger with the shop, so spend caps, cost matching, and duplicate-PO checks must run in code without LLM discretion.

Task: review

Re-validates proposals with validate_po_proposal: supplier and SKUs belong to tenant; active supplier; positive whole-number quantities; line count and total caps; unit cost matches product_suppliers within tolerance; no open PO already covering the SKU quantity. Does not create a purchase order itself.

6. Supplier Communication Agent

Use case

Draft supplier emails for order confirmation, chasing overdue POs, expediting, or delay notices. Triggered by /email and compound plans that include draft emails after scan/reorder.

Why it was added

Owners chase suppliers manually. This agent fills templates from PO facts (PO number, quantities, dates) so wording stays grounded. Email is never sent until the owner clicks Approve and Send.

Task: draft_emails

Uses get_po, get_supplier, get_exception, draft_email. LLM writes greeting, ask, and closing only. Code renders the template, checks draft_is_grounding (every PO number, quantity, date must appear in facts), retries once, then falls back to plain template. Inserts supplier_messages (draft) and a draft_email suggestion. Console mode (EMAIL_SENDER=console) logs only.

Email kinds

- order: new order or confirmation.
- chase: follow up on overdue or sent PO (default when supplier has open PO).
- expedite: request faster delivery.
- delay-notice: notify supplier of accepted delay.

Reply handling

POST /api/v1/supplier-replies stores inbound text. Extracted dates or delays become owner-approved suggestions to update PO expected date. chase_no_reply detector fires when no reply after follow-up days.

7. Explainer Agent

Use case

Owners will not delegate purchasing they cannot verify. The explainer answers why a suggestion or exception fired, and what would change under different assumptions, using stored evidence only.

Why it was added

Phase 5 explainability. Every decision should tie to forecast, on-hand, lead time, and supplier reliability figures that were stored when the decision was made.

Task: explain (/why, /explain)

Tools: get_suggestion, get_run_trace, get_evidence, get_history. Targets: suggestion id, exception id, or PO. LLM writes prose from typed evidence only. Post-check: every number and date in the answer must exist in evidence, else template fallback. Card includes confidence, what would change the decision, and collapsible evidence table. If evidence cannot answer, says so and lists missing data.

Task: whatif (/whatif)

Scenarios: supplier delay N days, demand up/down X percent, lead time change. Calls whatif_compare (replenishment formula with modified inputs, no LLM math). Returns whatif_compare card with before/after stockout date, recommended quantity, and cost. LLM narrates the difference only.

UI integration

- Why? button on PO suggestions, email drafts, exception cards, and Approvals tab.
- Insights Ask why box opens Inbox with the question.

8. Data Quality Agent (placeholder)

Registered for /quality but not implemented yet. Returns a typed not implemented yet result so orchestrator plumbing can be tested. Future scope: catalog and ledger consistency checks.

9. Slash Commands Reference

Type commands in Agent Inbox chat. You can combine several in one message for compound plans. Staff can run read-only commands; draft_po and draft_email are owner-only.

Command	Intent	Agent.task	Write	Role	Description
/stock [search]	get_stock	forecast.get_stock	No	Owner, Staff	Shows stock levels by SKU and category. Optional filters include category, status, and date range.
/forecast [sku]	forecast	forecast.forecast	No	Owner, Staff	Displays future demand forecasts for a specific SKU, including historical data for comparison.
/scan or /exceptions	scan_exception	exception_monitor.scan	No	Owner, Staff	Shows a list of scan exceptions, such as items not scanned or discrepancies between system and physical counts.
/reorder	reorder	replenishment.recommend	No	Owner, Staff	Provides suggestions for items to reorder based on current stock levels, lead times, and sales trends.
/draft po <supplier>	draft_po	purchase_order.draft	Yes	Owner only	Generates a draft purchase order for a specific supplier, including item details, quantities, and prices.
/email <supplier>	draft_email	supplier_communication.draft_email	Yes	Owner only	Drafts an email to a supplier, such as for order updates, inquiries, or notifications.
/explain or /why	explain	explainer.explain	No	Owner, Staff	Provides a detailed explanation for a specific data point or trend, such as a price change or demand spike.
/whatif <scenario>	whatif	explainer.whatif	No	Owner, Staff	Simulates the impact of a hypothetical scenario on inventory levels, demand, or costs.
/quality	data_quality	data_quality.check	No	Owner, Staff	Checks the quality and accuracy of inventory data, identifying missing or incorrect information.

10. Compound Requests and Free Text

Example: `/scan /reorder /email` or plain English: scan for problems, reorder what's needed and draft emails to the suppliers. The LLM proposes a DAG; code validates it. Typical compound flow: `exception_monitor.scan` and `replenishment.recommend` run in parallel; `supplier_comm.draft_emails` waits for both; `guardrail.review` runs last. Write steps require owner Run click on the plan card.

Name resolution

Supplier names, product names, and SKUs in commands or free text are resolved to tenant-scoped ids in code. The model never supplies `business_id` or cross-tenant ids.

11. UI Cards and Approval Flow

Each finished orchestrator step emits a typed card:

- `stock_table`, `forecast_chart`: forecast agent reads.
- `exception_list`: exception monitor.
- `po_suggestion`: replenishment draft with Approve/Reject and Why?.
- `email_draft`: editable subject/body, Approve and Send, Save draft, Reject, Why?.
- `explanation`: prose, confidence, evidence table, missing-data notice.
- `whatif_compare`: before/after metrics per SKU.
- `clarification / refusal`: orchestrator could not proceed safely.

Write plans show a plan card with Run / Edit / Cancel before any draft suggestion is stored.

12. Roles

- Owner: all commands including `/draft-po` and `/email`; approves suggestions and sends email.
- Staff: read-only agent commands (`/stock`, `/forecast`, `/scan`, `/reorder`, `/why`, `/whatif`); cannot draft POs or supplier emails.

13. Nightly Exception Scan

POST `/api/v1/jobs/exception-scan` (owner JWT or X-Job-Secret). Optional scheduler when `EXCEPTION_SCAN_SCHEDULER_ENABLED=true`. Owners set `exception_scan_enabled`, `exception_scan_hour_utc` (default 02:00 UTC), and `chase_followup_days` (default 3) in Settings autonomy rules.

14. Key Assumptions and Limits

- Forecasts use seasonal naive or daily average from 56 days of sales; no Prophet/statsforecast.
- Replenishment what-if delay/lead-time adds `daily_average` x extra days to needed quantity.
- Stockout date in what-if = `on_hand / daily_average` from today, or none if no demand.
- Tests use `LLM_PROVIDER=fake`; live LLMs are not called in pytest.
- `data_quality` agent is still a placeholder.
- Agents never post stock; only the ledger service can.

15. Detailed Command Walkthroughs

/stock

When to use

Before placing orders, after a big sale day, or when a staff member asks how much is on hand for a SKU or location.

Examples

/stock
/stock OATS-1
/stock oats

What happens

Orchestrator routes to forecast.get_stock. get_stock tool queries the inventory service (sum of ledger movements). You receive a stock_table card listing SKU, product name, on-hand, and low/out status per location.

Write?

No. Read-only for owner and staff.

/forecast

When to use

Planning purchases for the next two weeks, validating Insights numbers in chat, or checking one SKU's demand pattern.

Examples

/forecast
/forecast SKU-100
Why is demand up for olive oil?

What happens

forecast.forecast runs run_forecast and related tools. Default window: 56 days history, 14-day horizon. Methods: seasonal_naive (repeat last 7 days), daily_average (short history), no_sales (zeros). Card: forecast_chart with trend, chosen model, WAPE, confidence, caveats.

Write?

No. Same forecast service as Insights; does not create POs.

/scan and /exceptions

When to use

Start of day review, after receiving goods, or when something feels off. Nightly job runs the same detectors automatically.

Examples

/scan
/exceptions

What happens

exception_monitor.scan runs detectors, LLM ranks playbook actions, findings persist to exceptions table. Card: exception_list with severity, title, recommended_action, id for /why. May create generic suggestions for approval actions.

Write?

Orchestrator read-only (no plan pause). Persistence is internal to agent tools.

/reorder

When to use

Buying meeting: see which SKUs need stock and how many units, without committing a draft PO yet.

Examples

/reorder

reorder what's low

What happens

replenishment.recommend calls reorder_recommendations (formula in code), supplier reliability, open POs. LLM may filter SKUs or prefer alternate supplier when preferred is late. Text card with lines and evidence.

Write?

No. Staff and owner can run this.

/draft-po and /po

When to use

Owner wants the agent to propose a purchase order for a specific supplier after reviewing recommendations.

Examples

/draft-po Mill Co

/po Acme Supplies

What happens

Plan: recommend -> draft_po -> guardrail.review. Pauses for Run/Edit/Cancel. On Run, creates draft_po suggestion(s) with evidence. po_suggestion card in Inbox; Approvals tab lists pending items. Approve creates draft PO in Orders (still needs normal approve/send/receive flow).

Write?

Yes. Owner only. Requires plan approval and suggestion approval.

/email

When to use

Chase overdue PO, confirm a new order, expedite shipment, or send delay notice. Always review before send.

Examples

/email Mill chase

/email Acme order

/email Metro expedite
/email Acme delay-notice

What happens

Plan: recommend -> draft_emails -> guardrail. draft_email tool stores supplier_messages draft + suggestion. email_draft card: edit subject/body, Approve and Send (calls EmailSender), Save draft, Reject. Console mode logs only; SMTP mode sends after approval.

Write?

Yes. Owner only.

/why and /explain

When to use

Verify a PO suggestion quantity, understand an exception, or audit agent reasoning before approving.

Examples

/why suggestion <uuid>
/why exception <uuid>
/why did you suggest 200 units?
/explain

What happens

explainer.explain loads evidence from suggestion/exception/PO, run trace from agent_steps, optional sales history. LLM prose is grounded; bad numbers fall back to template. explanation card with evidence table, confidence, what would change the decision. Why? buttons on cards pre-fill /why commands.

Write?

No. Read-only.

/whatif

When to use

Sensitivity analysis before approving a large PO: what if demand rises or supplier is late?

Examples

/whatif demand up 20%
/whatif demand down 10%
/whatif supplier delay 5 days
/whatif lead time 14 days

What happens

whatif_compare reruns replenishment math with modified demand_pct, delay_days, or lead_time_days. whatif_compare card shows before/after stockout date, recommended qty, cost per SKU. LLM narrates only.

Write?

No. Read-only.

/quality

When to use

Future: catalog/ledger consistency checks. Not available yet.

Examples

/quality

What happens

data_quality.check placeholder returns not implemented yet text card.

Write?

No.

16. Agent Tool Registry

Read tools are tenant-scoped via `AgentContext.business_id` from the JWT.

Agent	Tools	Access
forecast	get_stock, get_forecast, get_history, run_forecast, get_forecast_accuracy	Read
exception_monitor	scan_detectors, record_exception_actions	Write suggestions/exceptions
replenishment	create_po_suggestions, get_supplier_reliability, get_open_pos,	Read + guarded write
supplier_comm	get_po, get_supplier, get_exception, draft_email	Write draft only
explainer	get_suggestion, get_run_trace, get_evidence, get_history, whatif_reorder	Read
guardrail	(validates proposals in code)	Read

17. Daily Owner Workflow with Agents

- 1) Morning: `/scan` to see exceptions; `/stock` and `/forecast` for hot SKUs.
 - 2) Planning: `/reorder` for quantities; `/why` on anything unclear.
 - 3) Action: `/draft-po` for suppliers (owner); approve suggestions in Inbox.
 - 4) Suppliers: `/email chase` for overdue POs; approve send when ready.
 - 5) What-if: `/whatif` demand up 15% before increasing order size.
 - 6) Overnight: nightly exception scan (if enabled) surfaces new inbox work.
- Every write path leaves `audit_log` and suggestion decision history.

18. SSE Events (what you see in chat)

- thinking - orchestrator node started.
- plan - DAG accepted; checklist appears.
- step_started / step_done / step_failed - per-agent progress.
- card - typed result (table, chart, suggestion, explanation, etc.).
- awaiting_approval - write plan paused for Run.
- token + done - streamed summary text, run complete.

19. Quick Command Cheat Sheet

- `/stock` - what do I have?
- `/forecast` - what will sell next 14 days?
- `/scan` - what is wrong?
- `/reorder` - what should I buy?
- `/draft-po Acme` - propose a PO (owner, needs approval)
- `/email Acme chase` - draft chase email (owner, needs approval)
- `/why suggestion <id>` - why was this suggested?
- `/whatif demand up 20%` - counterfactual reorder impact
- `/quality` - data checks (coming soon)

Regenerate this PDF: `python docs/generate_agents_guide_pdf.py` (requires `fpdf2` in backend venv).